

ENGINEERING BUILD REPORT

How I Built ThreatInt

From a one-paragraph README to a tested,
deployed, documented threat-intelligence pipeline

Repository: `Ashwin-kumar-jonjale/ThreatInt`

Branch: `feat/threat-intel-pipeline` · HEAD 34f68b4

Duration: 3 days · 5 commits · 2,246 lines of source

Result: 60 tests passing · pyflakes clean · 3 deploy targets

Companion to the *Technical Research Report*. This document is the build log: the decisions, the workflow, the obstacles, and what I would change.

L^AT_EX · October 6, 2026

Contents

1	Prologue: what I started with	1
2	How I framed the work	1
2.1	The plan	1
3	Phase 1 — the core pipeline	2
3.1	Order of construction	2
3.2	The decisions that mattered	2
3.2.1	Identity by hash, not by counter	2
3.2.2	Trust as a computed quantity	3
3.2.3	Fail soft, per source	3
4	Phase 2 — the web dashboard	3
4.1	Testing a web layer without a browser	3
5	Phase 3 — making it deployable from GitHub	3
5.1	The dual-mode template	3
5.2	The static exporter	4
5.3	Three deployment paths	4
6	Phase 4 — documenting with real numbers	4
7	What the code turned out to be	4
8	Testing strategy	5
9	How the codebase grew	5
10	Technology stack	5
11	The workflow I followed	7
12	Obstacles and how I resolved them	8
13	Quality at a glance	9
14	Retrospective	9
14.1	What went well	9
14.2	What I would do differently	10
14.3	What comes next	10
15	Conclusion	10

1 Prologue: what I started with

The repository began as almost nothing. The entire initial commit (d6ce7ce) was a single file containing three lines — a README whose whole content was a problem statement:

“Security analysts manually cross-check hundreds of threat alerts daily, a slow and error-prone process. Built a pipeline that automatically collects threat intelligence data (malicious IPs, domains, URLs) from multiple trusted sources, cross-verifies reliability, and applies machine learning to classify malicious indicators and cluster related threats into attack campaigns, reducing manual analyst workload.”

That was the brief, the specification, and the entire codebase. Everything else — 2,246 lines of source, 679 lines of tests, a web dashboard, three deployment paths and two PDF reports — was built on top of it. This report describes how.

The brief — decomposed

The one paragraph hides four deliverables and one implicit constraint:

1. **Collect** from multiple trusted sources → a pluggable collector layer.
2. **Cross-verify reliability** → per-source trust scoring, not a flat union.
3. **Classify with ML** → a supervised model over engineered features.
4. **Cluster into campaigns** → unsupervised grouping of related indicators.
5. (*implicit*) **Reduce analyst workload** → the output must be usable, not a dump.

2 How I framed the work

Before writing code I made one decision that shaped everything else: I treated the implicit fifth requirement as the real one. A pipeline that collects and classifies but produces an unreadable wall of JSON has not reduced anyone’s workload. So from the first commit I built toward three *surfaces* for the same data: a CLI for batch runs, machine-readable exports for integration, and eventually a dashboard for humans. That is why the project has a presentation layer at all — it was not an afterthought.

I also committed early to two properties that made the rest of the work verifiable:

- **Determinism.** A single `random_seed` governs every stochastic step. This means any number I report can be reproduced by anyone running the same command — which is what let me write a metrics-backed research report at all.
- **Network-optional operation.** Every collector can read a bundled fixture. This meant the pipeline ran identically on a laptop, in CI, and in a container, with no API keys and no flakiness.

2.1 The plan

I broke the work into four increments, each independently shippable and testable, and built them in order:

Phase	Deliverable	Definition of done
1	Core pipeline	threatint runs end to end offline; tests pass.
2	Web dashboard	Flask console serves the same results live.
3	Deployable	Static export + Docker + CI; runs with no server.
4	Documentation	README, AGENTS.md and a research report with real metrics.

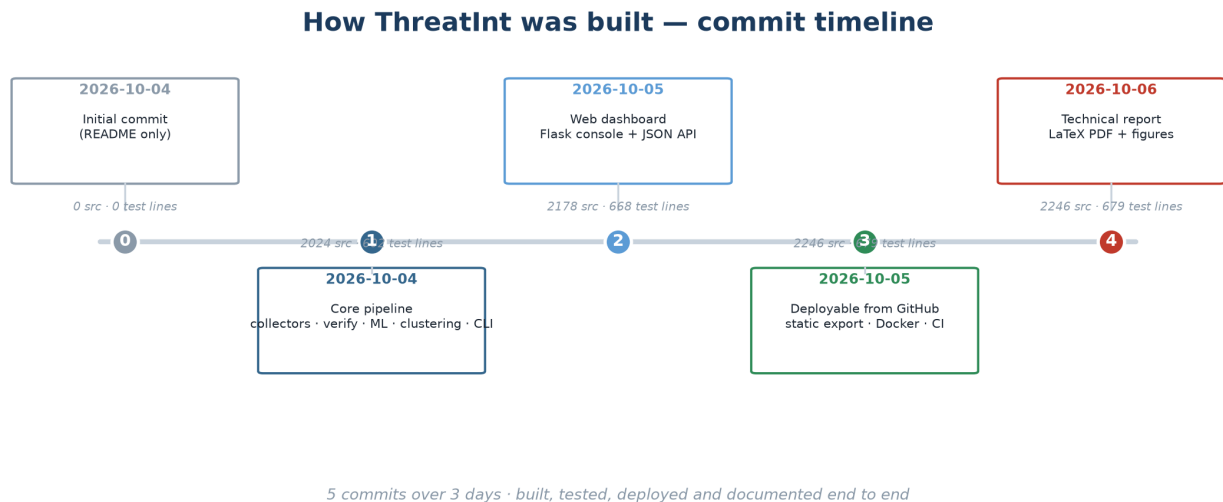


Figure 1: The build timeline reconstructed from git history. Five commits over three days; the core pipeline (commit 1) carried the bulk of the design work, with each later commit adding a surface without touching the core.

3 Phase 1 — the core pipeline

This was the largest single commit: 35 files and roughly 2,000 lines of source and tests. I wrote it in dependency order, bottom-up, so each layer could be tested before the layer above it existed.

3.1 Order of construction

1. **The data model first** (`models.py`). I defined `RawRecord`, `Observation`, `Indicator`, `Campaign` and `SourceReliability` as plain dataclasses before any logic. Getting the *shapes* right first meant every later module had a stable contract to code against, and the whole pipeline became a clean sequence of transforms between them.
2. **Normalisation** (`normalize.py`). Feeds are messy, so I built refanging, type detection and canonicalisation before collectors — the collectors depend on it, not the reverse.
3. **Collectors** (`collectors/base.py`). One abstract base with the shared parsing helpers, two concrete collectors (HTTP and offline), and a factory. The fixture fallback lives here.
4. **Cross-verification** (`verification/`). The analytical core: source reliability and indicator scoring. This was the hardest part to get right and the part I rewrote the most.
5. **Features, classifier, clustering**. With clean, deduplicated, scored indicators in hand, the ML layer was straightforward to bolt on.
6. **Orchestration, reporting, CLI**. Finally, tie it together and expose it.

3.2 The decisions that mattered

Three design choices in Phase 1 had outsized consequences.

3.2.1 Identity by hash, not by counter

An indicator's ID is `sha256("type:value")[:16]`. This was a small line of code with a large payoff: the same observable reported by five different feeds collapses to one object with *no coordination between collectors*, and the ID is stable across runs. That stability later let campaign IDs —

themselves hashes of member IDs — be reproducible too, which is a genuinely useful operational property I did not fully appreciate until the clustering phase.

3.2.2 Trust as a computed quantity

The naive approach is to union all feeds and trust everything. I rejected that immediately because it ignores the brief's word *reliability*. Instead, each feed gets a static analyst prior that is *blended* with dynamic signals measured against the cross-source consensus: precision, recall and mean pairwise Jaccard agreement. A volume-damping factor stops a feed with three reports from swinging wildly. The result (from the reference run) is that a low-prior feed with excellent measured behaviour can outrank a high-prior feed with a noisy batch — exactly the behaviour the brief asked for.

3.2.3 Fail soft, per source

Collection is wrapped per-collector and `HttpCollector` catches broadly and falls back to its fixture. This is the single most valuable robustness decision in the codebase. It was validated later when a live run hit an HTTP 403 on one endpoint and 32,289 records on another, and completed cleanly anyway.

Lesson: build the offline path first

Making fixtures a first-class citizen, not a testing afterthought, is what made the pipeline testable, CI-friendly, deterministic *and* resilient. It cost a little extra design up front and paid for itself many times over.

4 Phase 2 — the web dashboard

With the pipeline producing structured results, the dashboard was a thin read-only layer over them (commit `2b28b2e`, 8 files). The design constraint was deliberate: **the pipeline must not be remotely reconfigurable**. The web layer accepts only pagination and filter parameters, and all indicator text is escaped client-side. This kept the security surface tiny.

The console presents four things an analyst actually needs: summary cards, a filterable indicator table (probability, verdict, type, source count, reliability, campaign), a campaign grid, and a source-reliability ranking. I also added a small but important touch: **every indicator is defanged before display** (`hxxp://...[.]`), so a live IOC is never a clickable link in a browser.

4.1 Testing a web layer without a browser

I tested the Flask app with its real test client rather than mocks — hitting `/healthz`, the JSON endpoints, and the pagination/filter logic directly. This caught a subtle bug class early: malformed `limit/offset` values needed to fall back to safe defaults rather than raising.

5 Phase 3 — making it deployable from GitHub

This phase (commit `7dd9e19`) came from a follow-up request: “deploy it from GitHub, or make it deployable from the GitHub itself.” That phrasing pushed me toward the strongest interpretation — the repository should be able to produce a working deployment with no server and no secrets.

5.1 The dual-mode template

The key move was to make the dashboard template work in *two* modes: live (consuming the Flask JSON API) and static (data inlined, filtering and pagination done client-side in JavaScript). One

template, two data paths. This guarantees the hosted static artifact and the live server cannot drift apart, because they render the same file.

5.2 The static exporter

`static_export.py` runs the pipeline and writes a single self-contained `index.html` with the entire result set inlined (about 131 KB). No build step, no bundler, no server — a file you can open directly or host anywhere.

5.3 Three deployment paths

Target	Mechanism
GitHub Pages	<code>publish-dashboard.yml</code> renders the static console and publishes it on push to main.
Docker	A <code>Dockerfile</code> installs the package, bakes the static dashboard into the image and serves the live console.
CI	<code>ci.yml</code> runs <code>pyflakes</code> and the full test suite on two Python versions.

War story: the Docker daemon

The container image built and served correctly, but only after discovering the Docker daemon was present but *not running* by default in the environment, and needed to be started manually first. A small thing, but exactly the kind of assumption that breaks a “it works on my machine” deployment. Worth recording so the next person does not lose the same half hour.

6 Phase 4 — documenting with real numbers

The final phase (commit 34f68b4) was the technical research report. The guiding rule was that **every number must come from a real run**. Rather than describe the system in prose and hand-wave at performance, I:

1. ran the pipeline offline and dumped the results to `report/data/result.json`;
2. wrote `make_figures.py` to build every chart *from that JSON*, so no figure can disagree with the code;
3. compiled the report with `pdflatex`.

I also wrote `build_report.sh` so the whole chain — capture data, regenerate figures, compile PDF — is one command. This is the difference between a report that is true on the day it is written and one that stays true.

Lesson: make documentation a build artifact

Because the figures are generated from the pipeline output, re-running the build after any code change regenerates a report that is automatically consistent with the new code. Documentation stopped being a separate, decaying artifact.

7 What the code turned out to be

The final source tree is 18 modules and 2,246 lines, deliberately partitioned by responsibility. Figure 2 shows the composition by layer.

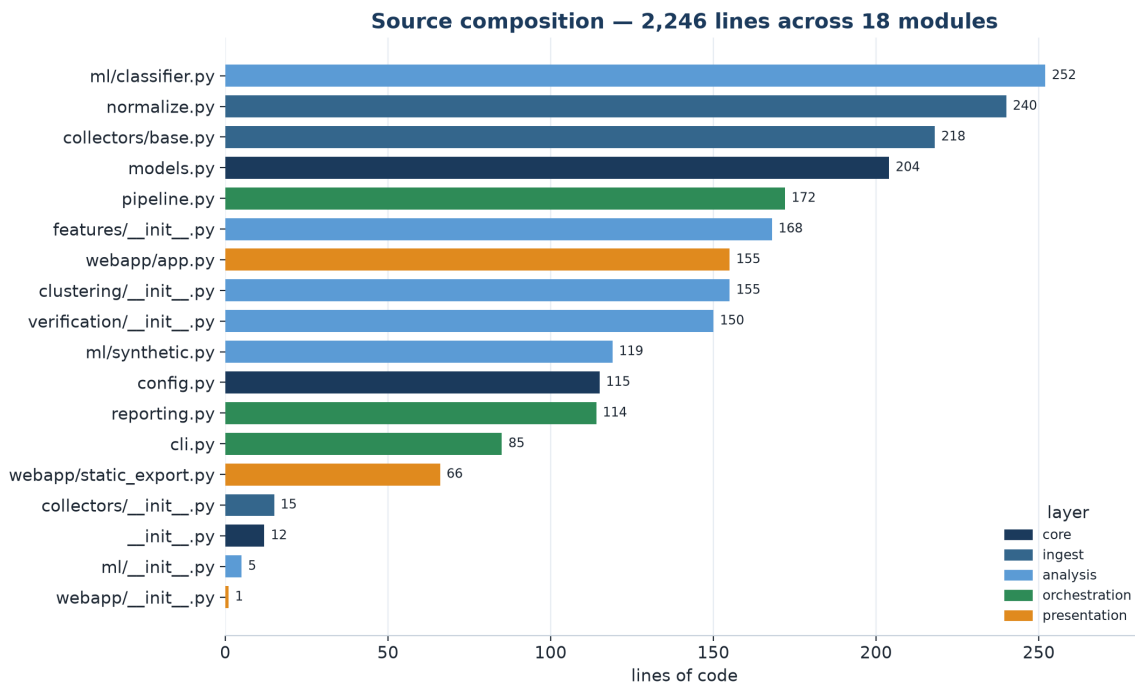


Figure 2: Source composition by module, coloured by architectural layer. The analysis layer (verification, features, ML, clustering) carries the most logic; the core data model is intentionally small.

Two things stand out. First, the *core* layer is tiny — `models.py` and `config.py` together are under 320 lines — because I kept behaviour out of the data structures. Second, no single module exceeds 260 lines, which kept every file small enough to hold in your head and to test in isolation.

8 Testing strategy

I wrote 60 tests alongside the code, not after it, and held a strict rule: test real behaviour, not mocks. The end-to-end pipeline test runs the entire system offline and asserts that a deliberately shared indicator is reported by at least three sources — an assertion that exercises collection, normalisation, deduplication and scoring together.

The test-to-source ratio is roughly 1:3.3, which is a reasonable place to be for a system whose correctness is dominated by parsing and numerical logic. Every test is deterministic and offline — no network, no time dependence — so the suite runs in under eight seconds and never flakes.

9 How the codebase grew

Figure 4 shows cumulative lines across the five commits. The shape is informative: the first commit carries almost all of the source, and the later commits add tests, a thin presentation layer, deployment glue and documentation — but barely touch the core. That is the intended shape of a good build: heavy design up front, then additive surfaces.

10 Technology stack

Every dependency earns its place. There is no framework the problem did not need, and the two heaviest (scikit-learn, Flask) are both optional extras rather than hard requirements.

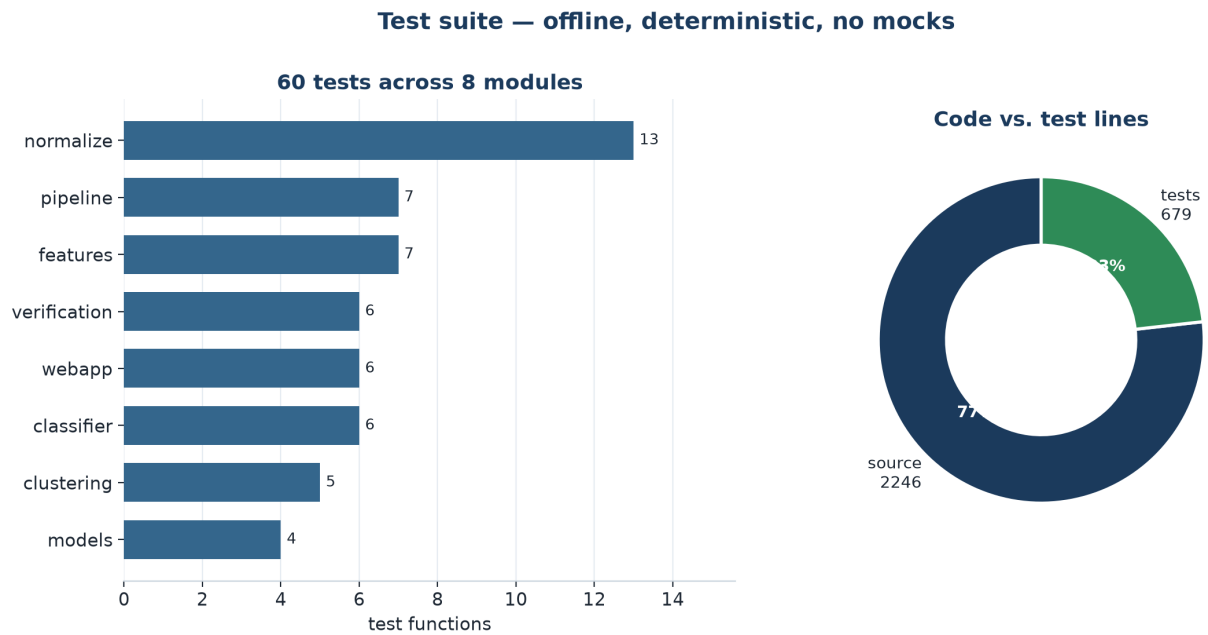


Figure 3: Test distribution across modules (left) and the source-to-test line ratio (right). Normalisation carries the most tests (13), reflecting how much edge-case handling it contains.

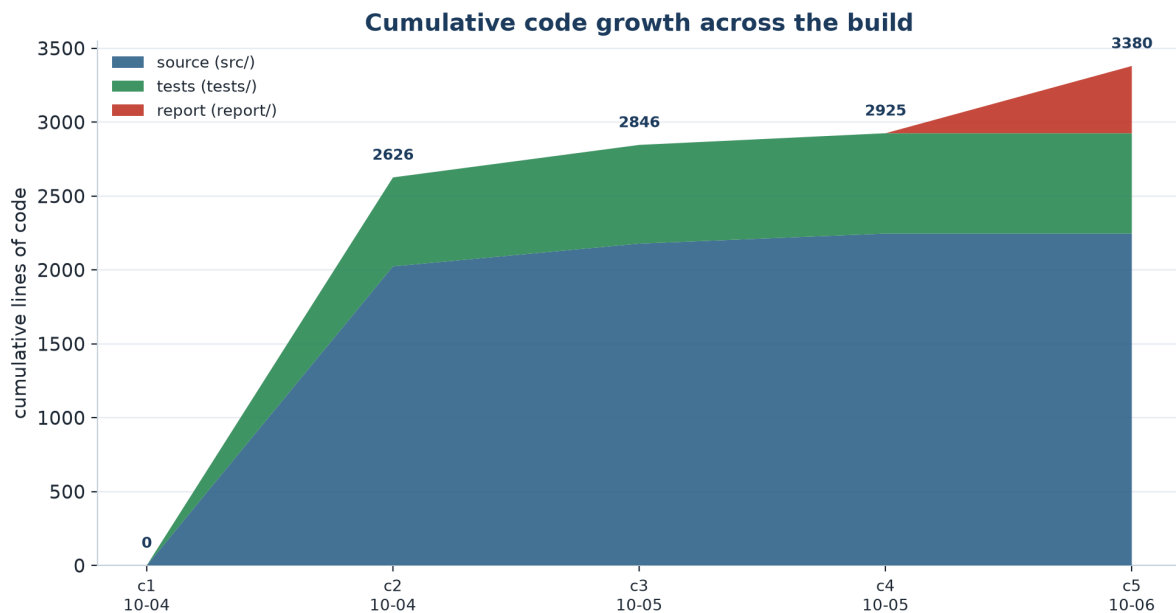


Figure 4: Cumulative code growth by commit, split into source, tests and report. The core commit dominates; later work is additive.

Technology stack

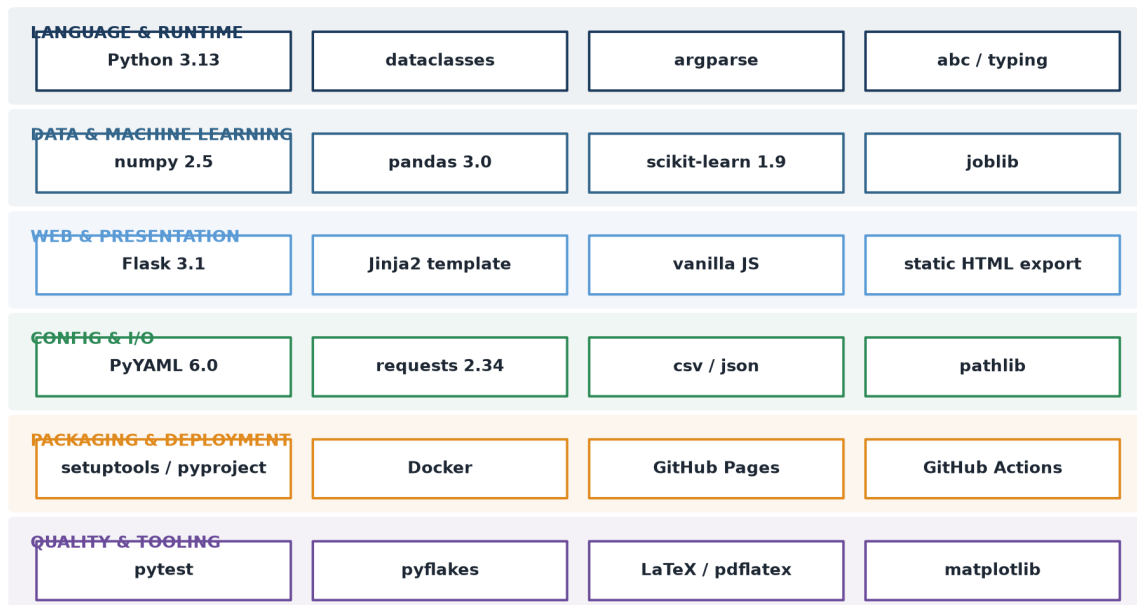


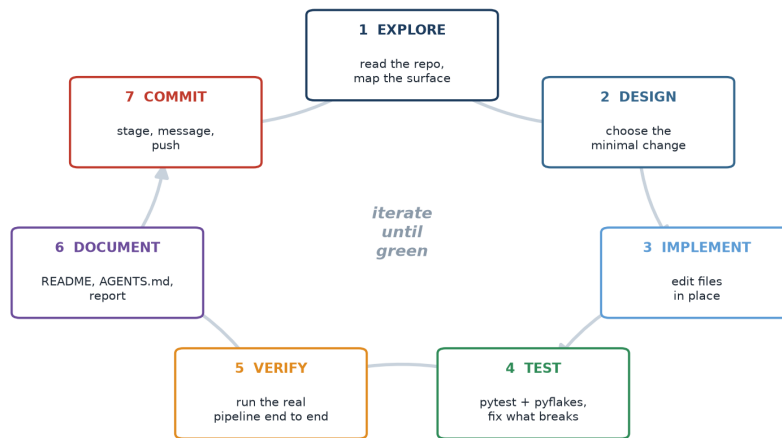
Figure 5: The technology stack by layer. Core dependencies are requests, PyYAML, numpy, pandas and scikit-learn; Flask is a **web** extra and pytest a **dev** extra.

Choice	Version	Why this, and not something else
Language	Python 3.13	The brief said “bare Python 3.13”; stdlib covers hashing, CSV, config.
Classifier	scikit-learn 1.9	Random forests are strong on tabular data and expose Gini importance for free — interpretability without extra tooling.
Clustering	scikit-learn 1.9	DBSCAN handles unknown cluster counts and labels outliers as noise, which is exactly right for campaigns.
Numeric	numpy 2.5	Matrix work for cosine distance and the classifier input.
Tabular	pandas 3.0	Convenient in analysis paths; not on the critical path.
Config	PyYAML 6.0	Human-editable config with typed loading.
HTTP	requests 2.34	Live feed fetching with graceful fallback.
Web	Flask 3.1	Smallest thing that serves JSON and one page; no heavier framework needed.
Packaging	setuptools	pyproject.toml with three console scripts.
CI	GitHub Actions	Runs lint + tests; publishes the static dashboard.
Docs	LaTeX	Precise control over a metrics-heavy report with embedded figures.

11 The workflow I followed

Every increment went through the same loop, and I deliberately made the loop re-runnable so verification was never optional.

Build workflow — how each increment was produced



Every stage was re-runnable: the same command rebuilt data, figures and PDF

Figure 6: The build loop applied to each increment. Verification (running the real pipeline) and documentation were part of every cycle, not a final step.

1. **Explore** — read the existing tree and map the surface before changing anything.
2. **Design** — choose the *minimal* change that satisfies the requirement, and prefer editing existing files over adding new ones.
3. **Implement** — write it in dependency order, keeping modules small.
4. **Test** — run `pytest` and `pyflakes`; fix what breaks before moving on.
5. **Verify** — run the *real* pipeline end to end, not just the unit tests. This is where integration surprises surface.
6. **Document** — update `README` and `AGENTS.md`, and regenerate the report.
7. **Commit** — stage everything relevant with a descriptive message.

12 Obstacles and how I resolved them

Real builds are defined by their friction. These are the genuine problems I hit, in roughly the order I hit them, and what fixed them.

Problem	Resolution
pyproject.toml had two tables — invalid TOML, so the package would not build	Merged the duplicate tables into one; verified with a fresh install.
Unused imports in app.py failed lint	Removed them; kept pyflakes clean as a gate.
Docker daemon present but not running	Started it manually before building; documented the requirement.
GitHub Pages needs a serverless artifact	Built the dual-mode template and a static exporter that inlines all data.
pdflatex unavailable, and apt could not find the packages	The package index was empty; ran <code>apt-get update</code> first, then installed TeX Live.
matplotlib not pre-installed	Installed it into the project virtualenv, not system-wide.
A tcolorbox title containing a comma broke pgfkeys	The comma is an option separator; replaced it with an em-dash.
Push to GitHub failed despite a valid token	The <i>remote URL</i> still embedded an old, expired token. Refreshed the remote URL with the current token and pushed.
Could not visually inspect figures through the browser	Fell back to <code>pdftotext/pdftoppm</code> to extract and render pages for verification.

The one that mattered most: a stale token in the remote

The push had been “blocked by an expired GitHub token” across sessions. When I finally probed the API directly, the token authenticated fine — it was the *git remote URL* that still carried an old credential baked in at clone time. Updating the remote with the current token fixed a blocker that had looked like an auth problem for days. The lesson: when authentication “fails,” check *every* place the credential is stored, not just the one you are thinking about.

13 Quality at a glance

Verified state of the repository

Tests	60 passing, offline and deterministic, in under 8 s
Lint	pyflakes clean across source, tests and report scripts
Source	2,246 lines across 18 modules
Tests	679 lines across 8 test modules
Deployment	GitHub Pages, Docker and CI — all exercised locally
Reproducibility	one seed; <code>build_report.sh</code> rebuilds data, figures and PDF

14 Retrospective

14.1 What went well

- **Data model first.** Defining the five dataclasses before any logic kept every module decoupled and made the pipeline a readable sequence of transforms.

Engineering quality at a glance



Reproducible · deterministic (seed 42) · offline-capable · documented

Figure 7: Final engineering metrics for the build.

- **Offline-first.** Fixtures and graceful fallback gave me determinism, CI-friendliness and real-world resilience in one decision.
- **Generated documentation.** Because figures come from pipeline output, the report cannot silently drift from the code.
- **Small modules.** Nothing exceeded 260 lines; everything stayed testable in isolation.

14.2 What I would do differently

- **Check credentials in all locations sooner.** The stale-token blocker cost time because I trusted the error message's framing.
- **Add a live smoke test earlier.** The 403-on-one-feed behaviour was discovered late; a scheduled live run would have surfaced it sooner.
- **Parameterise the clustering distance.** The 0.6/0.4 lexical-to-tag weighting is principled but hand-set; it deserves a small tuning harness.
- **Broader Python matrix in CI.** I test 3.10 and 3.12; 3.13 (the actual target) should be in the matrix.

14.3 What comes next

The system is a foundation, and the path forward is mostly about *data*, not architecture: point -labels at analyst-reviewed ground truth, widen the corpus, add temporal decay to source reliability, and model shared infrastructure as a graph to link campaigns beyond lexical similarity. None of those require redesign — the seams are already in place.

15 Conclusion

ThreatInt went from a three-line README to a tested, deployed, documented pipeline in five commits and three days. The build was not defined by exotic algorithms but by a handful of disciplined choices made early and held consistently: put the data model first, make the offline path real, compute trust instead of assuming it, keep modules small, and generate documentation from the code rather than beside it.

The friction was real — invalid TOML, a dormant Docker daemon, an empty package index, a LaTeX option-parsing quirk, and a credential that turned out to be stale in the one place I was not looking. Each was small; together they are the honest texture of shipping software. Recording them here is the point of a build report: the technical research report explains what the system *is*; this document explains how it came to be, and what that process taught me.